Bitácora Los Cuatro 01

[Descripción de la aplicación, describiendo contexto, identificando el problema, e introduciendo la solución]

1) Organización del equipo

Lista de los miembros del equipo

Miembro	Email	Cuenta github
Danilo Arce	danilo.arce@alumnos.uach.cl	LeNylon
Sebastián Burgos	sebastian.burgos01@alumnos.uach.cl	Louwz777
Franco Navarrete	franco.navarrete@alumnos.uach.cl	FrancoNavarreteN
Juan Romero	juan.romero01@alumnos.uach.cl	MrCeliako
Sebastián Vallejos	sebastian.vallejos@alumnos.uach.cl	SebastianVallejos01

Horarios de reuniones semanales

- Reunión con Mandante
- Reunión de equipo

Repositorio GitHub del proyecto

https://github.com/FrancoNavarreteN/TALLER4.git

2) Descripción general

Contexto y problema

Existen varios métodos de estudio usados por los estudiantes, uno de los más populares y comúnmente usados es el método Pomodoro, este método consiste en ponerse una marca de tiempo tanto para descanso y estudio, así uno puede absorber información y luego poder descansar haciendo otra cosa para distraerse, por ejemplo, estudiar 50 minutos y descansar 10 minutos.

Hay gente que usa el cronómetro del celular dependiendo de cuánto tiempo quieren usar, el tema es que es un poco tedioso estar yendo al cronómetro cada cierto tiempo, aparte que los estudiantes usualmente no saben cuánto progreso llevan, entonces hacen pomodoro hasta que logren su meta diaria.

Solución propuesta

La idea es fomentar, mediante un sistema práctico para organizar las horas de estudio, el hábito de estudio constante/diario.

Lo que pensamos es en un sitio web dirigido al método Pomodoro, en el cuál el usuario podrá personalizar los tiempos a su conveniencia y de paso también poder tener alarmas personalizadas cuando termine el tiempo de estudio, y los usuarios podrán implementar sus objetivos, ya sea estudiar, hacer una tarea, etc, con el objetivo de que puedan saber cuanto les faltaria y cuánto tiempo deberán estar con pomodoro.

Además implementar gráficos interactivos para visualizar el progreso dentro del día, semana o mes.

También queremos implementar conexión o vinculación con la aplicación de música que estén usando, como Spotify, Apple Music, etc, así puedan ahorrarse tiempo y puedan ordenar sus playlist que estén escuchando.

3) Historias de Usuario

ID	Historia de Usuario
HU1	COMO [usuario] QUIERO [qué desea poder hacer o ver con la aplicación] PARA [objetivo *real* que quiere alcanzar]

4) Product Back-log

Las tareas deben ser las más atómicas o precisas posibles (factible en aproximadamente 2 o 4 horas). Por ejemplo: “Crear un botón “OK” para validar el formulario”. Ver más ejemplos abajo. Asignar uno o varios responsables por cada tarea.
Marcar el estado de las tareas con colores de fondo para las filas:

	Sin comenzar
	Asignada para la semana
	Avanzada, pero no terminada
	Terminada

ID	Descripción	Estimación (hh)	Responsable	Sprint #

HUx	COMO … QUIERO … PARA			
Tx.1	Diseñar/Implementar/Probar/Aprender/Desplegar/etc. ...			
Tx.2	Diseñar/Implementar/Probar/Aprender/Desplegar/etc. ...			

Asignar uno o varios responsables por cada tarea.

Ejemplo de tareas

ID	Descripción	Estimación (hh)	Responsable	Sprint #
H1	Como usuario quiero ver el hilo de mensajes de un canal para estar al tanto de lo que ocurre			
T2.1	Realizar tutorial de next y react para aplicaciones Web			
T2.2	Construir una página de ejemplo a partir del ejemplo visto en tutorial			
T2.3	Confeccionar mockup de baja resolución de la pantalla principal de la lista de mensajes			
T2.4	Seleccionar componentes chakra para layout de la aplicación			

T2.5	Implementar layout del mockup en prototipo semifuncional			
T2.6	Definir estructura de datos en *json* para mensajes de un canal y poblar con datos de ejemplo			
T2.7	Confeccionar mockup de alta resolución de cómo debe desplegarse un mensaje			
T2.8	Implementar componente react para desplegar mensaje			
T2.9	Integrar componente de mensaje en prototipo página (prototipo funcional 2)			
T2.10	Realizar pruebas de la página de lista de mensajes incluyendo condiciones de borde (cuando hay muchos mensajes o cuando el canal está vacío, sin mensajes)			
T2.11	Desplegar la página de mensajes			

5) Minutas de Reunión

Por cada reunión del equipo, registrar participantes, temas discutidos o tareas realizadas, identificar acuerdos tomados o retroalimentación recibida.

Reunión DD/MM/YYYY

Participantes	
Temas/Tareas	
Retroalimentación	
Acuerdos	
Otras notas	

6) Artefactos de Diseño

Incluir aquí mockups, diagramas de componentes o partes del sistema, definición de servicios, modelo de datos, etc.

API

endpoint	app/api/mensajes
GET	parametros: userId retorna: { userID: String, mensajes: [{ mensajeId: String, fecha: Date, texto: String, replyTo: String },

	] }

POST	Parámetros:
	body:
	{
	texto: String,
	userId: String
	replyTo: String
	}
	
	retorna:
	{
	mensajeId: String
	}